

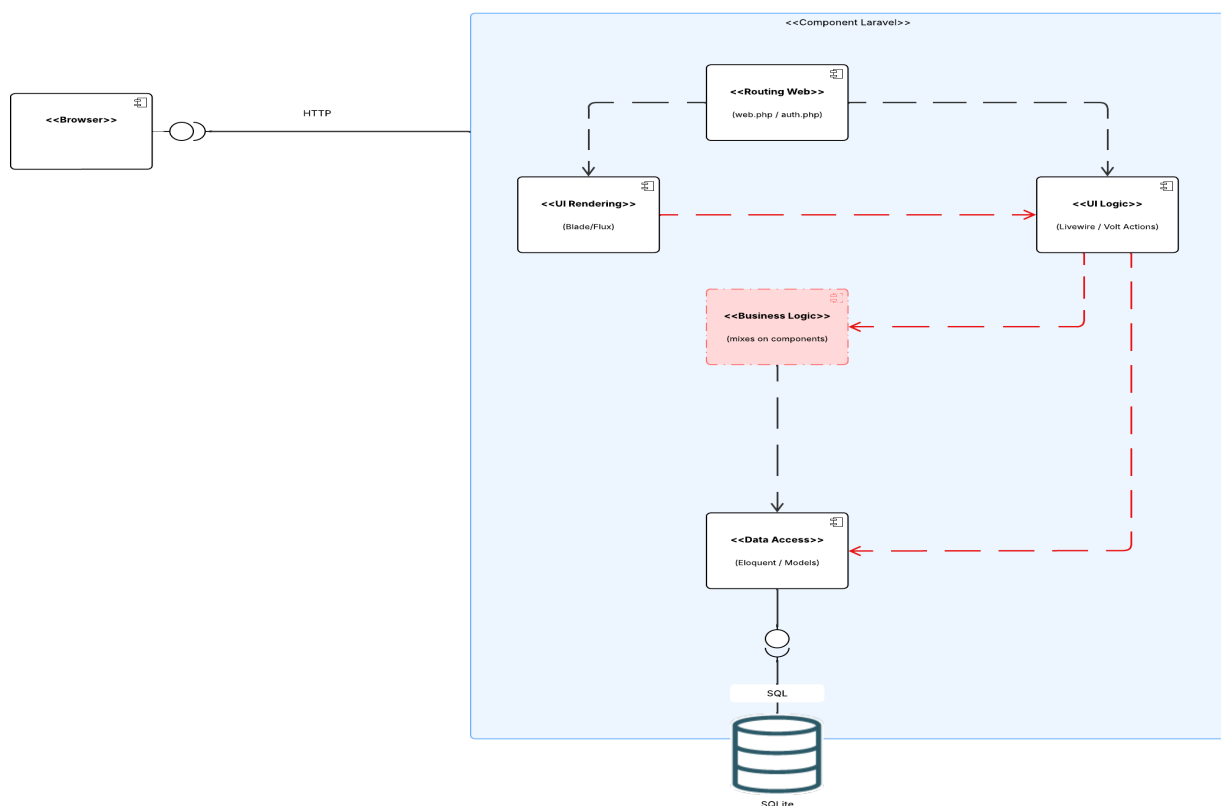
Documentation d'architecture

Cette documentation servira à décrire les choix d'architecture ainsi que les spécifications de l'API REST.

1. Introduction

Renote a été migrée d'un monolithe server-driven (Blade/Livewire) vers une architecture client-driven avec React, via un découplage complet front/back. Le back a été refactorisé en MVC/SOLID avec une API REST stable, et le front s'appuie désormais sur un state management pour centraliser l'état, les actions et les appels API.

2. Architecture de base

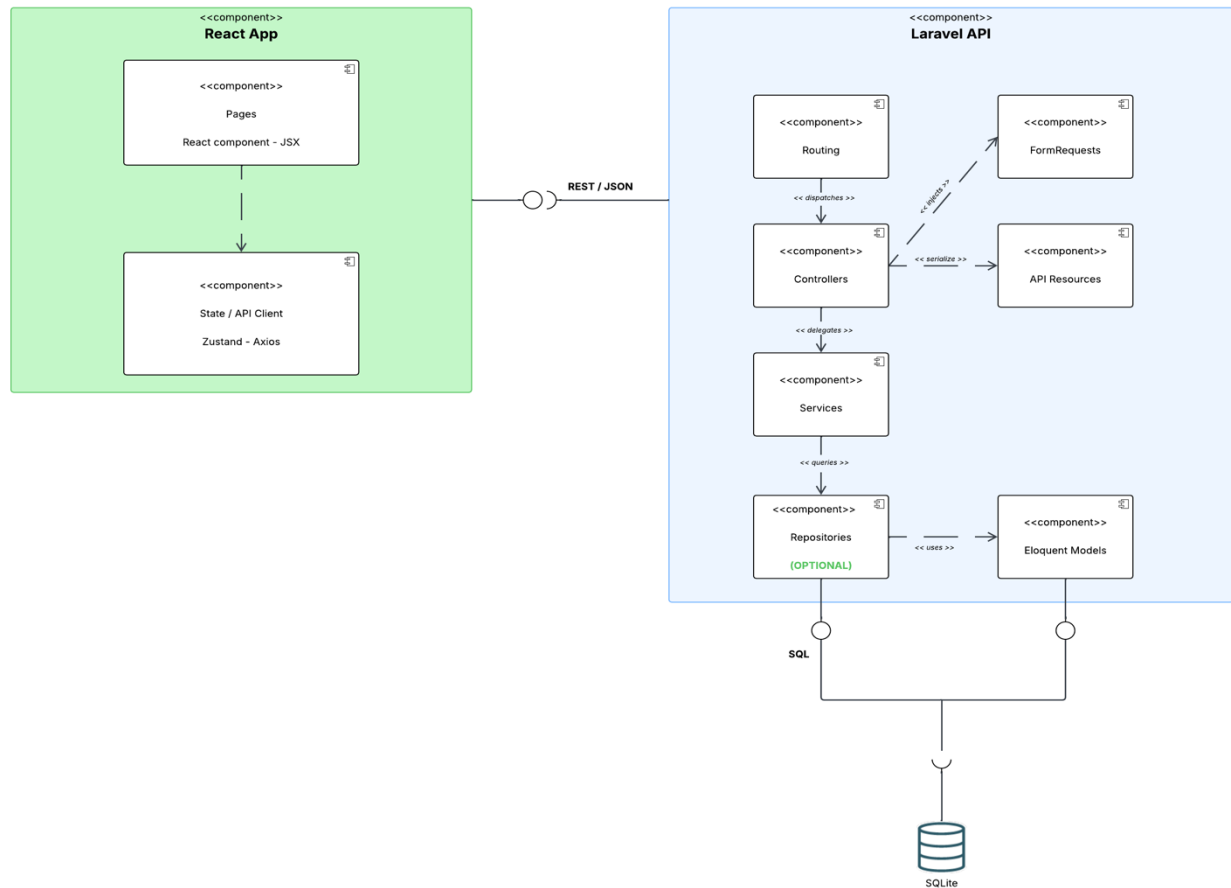


L'architecture initiale (monolithe Laravel + Livewire) expose plusieurs inconvénients :

1. **Couplage fort** : L'UI, la logique métier et l'accès aux données sont mélangés dans les composants Livewire. Un composant peut effectuer simultanément du traitement métier, de la validation, de la persistance et du rendu UI, ce qui viole le principe de responsabilité unique.
2. **Absence d'API REST** : Les routes reposent exclusivement sur des routes Web liées à Livewire, rendant l'application inaccessible depuis un client externe (mobile, SPA, service tiers).
3. **Mélange des responsabilités** : Sans couche service dédiée, la logique métier est dupliquée ou dispersée entre les composants, rendant le code difficile à maintenir.

3. Architecture cible

□. Composants d'architecture



Back-end

- **Controllers API** : Reçoivent les requêtes HTTP, délèguent aux services, retournent des réponses JSON standardisées
- **Services** : Contiennent la logique métier, injectés via leurs interfaces (IoC)
- **API Resources** : Transforment les modèles Eloquent en JSON, évitent l'exposition de données sensibles
- **Form Requests** : Valident et autorisent les requêtes en amont des controllers
- **Laravel Sanctum** : Gère l'authentification via tokens Bearer
- **Eloquent ORM** : Driver d'accès à la base de données SQLite via PDO

Front-end

- **React** : Rendu UI et navigation client-side
- **Zustand** : State management centralisé
- **Axios** : Client HTTP pour les appels API

Communication

- Protocole HTTP/REST entre React et Laravel
- Format JSON pour tous les échanges
- Authentification par token Bearer (Sanctum)

□. Spécifications API REST

Auth

Méthode	URL	Description	Auth
POST	/api/auth/login	Authentification	Non
POST	/api/auth/register	Création de compte	Non
POST	/api/auth/logout	Déconnexion	Oui

Password

Méthode	URL	Description	Auth
POST	/api/password/forgot-password	Envoi du lien de reset	Non
POST	/api/password/reset	Reset du mot de passe	Non
POST	/api/password/confirm	Confirmation du mot de passe	Oui
POST	/api/password/update	Mise à jour du mot de passe	Oui

Profile

Méthode	URL	Description	Auth
GET	/api/profile/me	Récupération du profil	Oui
PUT	/api/profile	Mise à jour du profil	Oui
DELETE	/api/profile	Suppression du compte	Oui

Note

Méthode	URL	Description	Auth
GET	/api/note	Liste des notes	Oui
POST	/api/note	Création d'une note	Oui
DELETE	/api/note/{id}	Suppression d'une note du compte	Oui

Tag

Méthode	URL	Description	Auth
GET	/api/tag	Liste des tags	Oui
POST	/api/tag	Création d'un tag	Oui

Exemple API :

```
1 // POST /api/auth/login
2 // Request
3 { "email": "user@example.com", "password": "password" }
4 // Response 200
5 {
6   "status": "success",
7   "message": "Login successful",
8   "data":
9     {
10      "user":
11        {
12          "id": 1,
13          "name": "John",
14          "email": "user@example.com"
15        },
16      "token": "1|abc123..."
17    }
18 }
19 // Response 401 - Invalid credentials
20 // Response 422 - Validation failed
```

□. Data flow du state management

1. **Action utilisateur** : ex: clic sur "Créer une note"
2. **Zustand action** : `createNote(text, tagId)` appelée depuis le composant
3. **Effet API** : Axios envoie `POST /api/notes` avec le token Bearer
4. **Réponse Laravel** : `JSON { status, message, data }`
5. **Update store** : La note est ajoutée dans `notes[]` du store Zustand
6. **Re-render React** : Les composants abonnés au store se mettent à jour automatiquement

4. Analyse de l'écart

Composants modifiés ou supprimés

- Les composants Livewire/Volt sont remplacés par des composants React — la logique UI et le rendu migrent côté client
- Le routing Web Laravel est remplacé par un routing API REST pur
- Les vues Blade ne servent plus qu'à servir le point d'entrée React (`index.html`)

Composants ajoutés

- Application React avec routing client-side
- Store Zustand pour la gestion d'état
- Controllers API dédiés (AuthController, NoteController, TagController, ProfileController, PasswordController, EmailController)
- API Resources (UserResource, NoteResource, TagResource)
- Form Requests pour la validation des entrées API

- Laravel Sanctum pour l'authentification par token

5. Justification de l'approche architecturale

Laravel Sanctum : Choisi pour sa simplicité d'intégration avec Laravel et sa gestion native des tokens API. Adapté pour une SPA qui consomme sa propre API.

Zustand : Préféré à Redux pour sa légèreté et sa syntaxe minimaliste. Pour une application de la taille de Renote, Redux introduirait une complexité inutile (boilerplate, actions, reducers). Zustand permet de définir un store en quelques lignes tout en restant scalable.

SOLID : Les interfaces (`AuthServiceInterface`, `NotesServiceInterface`, etc.) permettent d'inverser les dépendances (DIP) et de rendre chaque composant remplaçable sans modifier le reste du code (OCP). Les services ont une responsabilité unique et ne connaissent pas la couche HTTP (SRP).

API Resources : Garantissent qu'aucune donnée sensible (`password`, `remember_token`) n'est exposée accidentellement. Elles constituent un contrat stable entre le back et le futur front React.

Format de réponse standardisé : Le trait `ApiResponse` centralise le format `{ status, message, data }` sur tous les endpoints, facilitant la gestion des réponses côté Axios/Zustand.